A

AskMyDocs

RAG Document Q&A — grounded answers with citations

Part 1 of 2 — **Product & Features Walkthrough**

The upload → retrieve → answer pipeline, explained for interview prep.

Next.js 16 · PostgreSQL + Prisma · transformers.js embeddings · RAG · Claude → Gemini · streaming

Prepared 14 July 2026 · Author: Bhanu Prakash

0 · What AskMyDocs is

Upload PDFs / manuals / contracts, ask a plain-English question, and get an answer grounded **only** in your files — with per-source citations (document + page + match score). The whole point is trust: no hallucinations, every claim traceable to a passage.

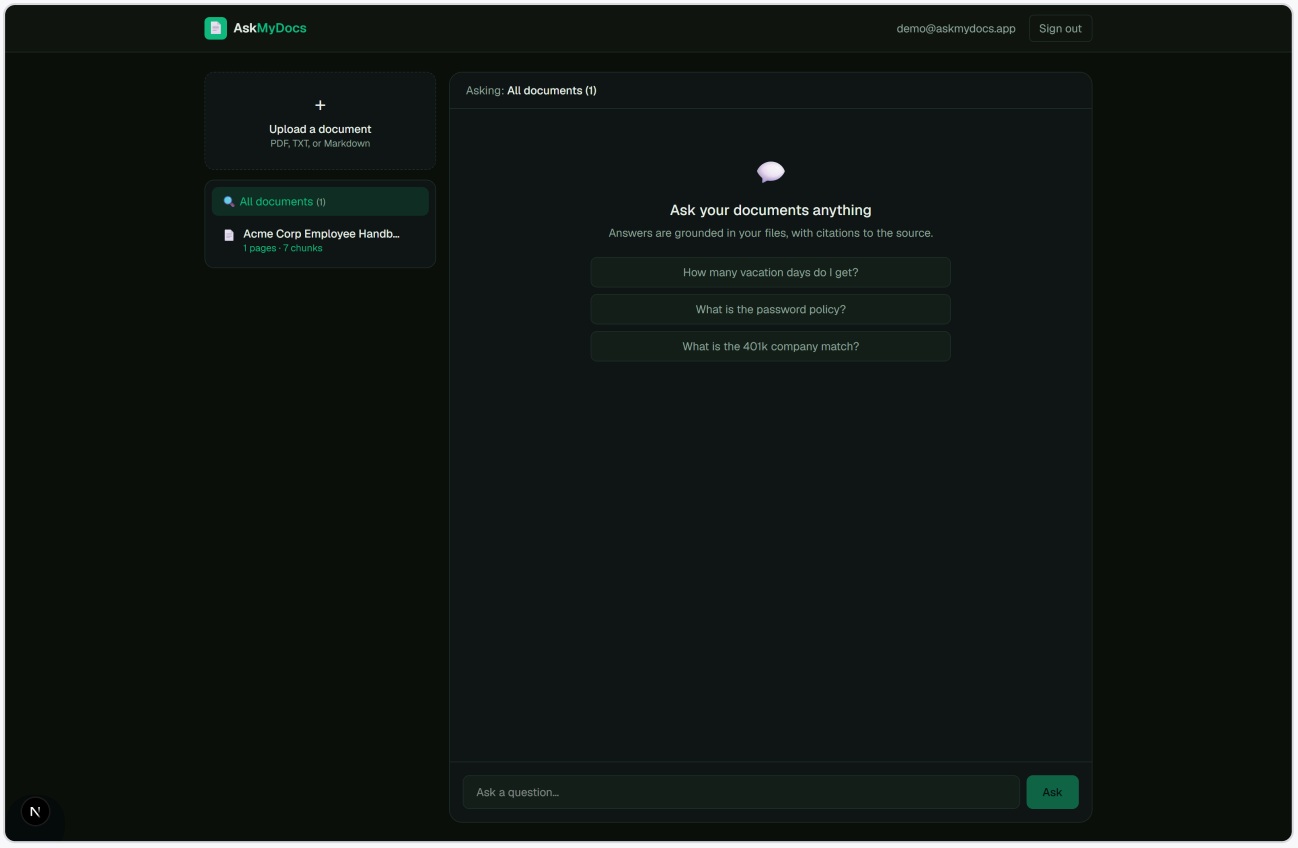
The interviewer one-liner: "It's a Next.js RAG app. Documents are parsed per-page, structure-aware chunked, and embedded *locally on-device* with transformers.js (all-MiniLM-L6-v2, 384-dim) stored as JSON vectors in Postgres. A question is embedded and matched by in-memory cosine similarity (top-5), then a grounded answer is streamed with a three-tier provider strategy (Claude → Gemini → extractive) and inline [n] citations — using a custom wire format that renders the sources before the answer."

Technology stack

LAYER	CHOICE
Framework / UI	Next.js 16.2.10 (App Router, streaming) · React 19 · TypeScript · Tailwind 4
Database / ORM	PostgreSQL · Prisma 6.19.3 (384-dim vectors stored as a JSON column)
Embeddings	**Local, on-device** — `@xenova/transformers`, model `Xenova/all-MiniLM-L6-v2`
PDF parsing	`pdf-parse` v2 (per-page, on pdfjs)
Answer generation	Claude `claude-sonnet-5` → Gemini `gemini-flash-latest` → extractive fallback
Auth	NextAuth v4 (Credentials, JWT) · bcryptjs

> **No external vector DB.** Vectors are stored as a JSON column on the `Chunk` table and searched with in-memory cosine similarity. The README flags a native `VECTOR`/ANN index as the scaling path — a good, honest "what I'd do next".

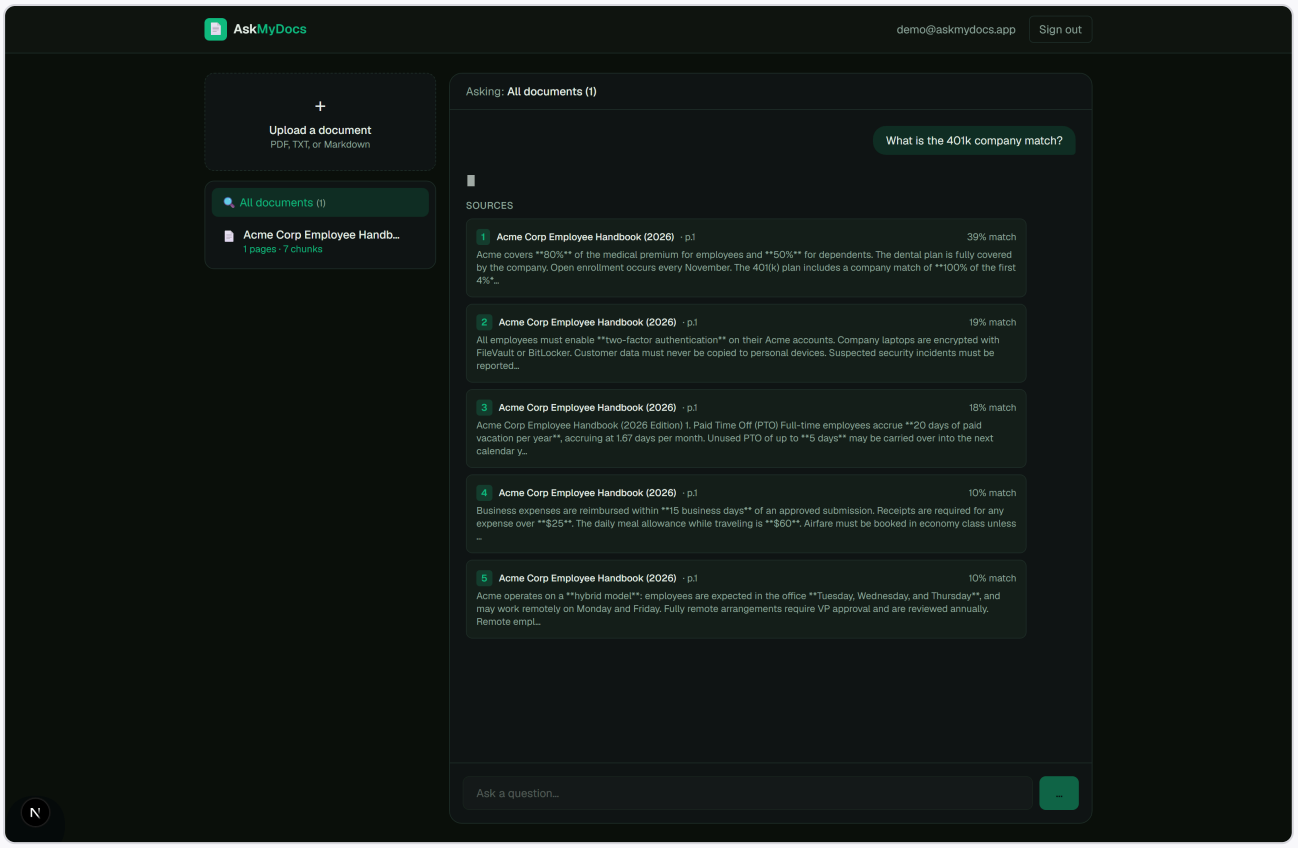
1 · The workspace



The chat workspace — document library (Acme Corp Employee Handbook · 1 page · 7 chunks), scope selector, and suggested questions

- **Document library** (left) — upload PDF/TXT/Markdown (15 MB max), see each document's page + chunk counts and status (PROCESSING → READY / FAILED), pick a scope ("All documents" vs a single doc), delete.
- **Chat panel** (right) — ask anything; suggested starter questions appear when a document is loaded.

Retrieval with citations — the visible proof



A question resolves to a ranked SOURCES panel — top-5 chunks, each with document, page, and % match — and the grounded answer streams above it with inline [n] badges

Each answer carries a **Sources** panel: the top-5 retrieved chunks, each showing the document title, page number, a **% match** score, and a snippet. The generated answer streams token-by-token above the sources and cites them inline as superscript `[n]` badges. In the capture the retriever correctly surfaces the 401(k) match ("100% of the first 4%") and PTO ("20 days of paid vacation") passages as the top hits.

2 · The RAG pipeline (the headline)

> **Two AI stages: local embeddings, then remote generation.**
> **Embeddings** (`src/lib/embeddings.ts`) — `Xenova/all-MiniLM-L6-v2` , 384-dim, run **in-process, offline, zero-cost** via `@xenova/transformers` (downloaded once, cached). Mean-pooled + L2-normalized and batched in groups of 32; because vectors are normalized, cosine reduces to a dot product.

Ingestion (`src/lib/ingest.ts`)

- **Parse** (`parse.ts`) — PDFs page-by-page (so citations map to real page numbers); TXT/MD as a single page.
- **Chunk** (`chunk.ts`) — structure-aware: split into markdown-heading/paragraph blocks, pack into ~450-char chunks, hard-split oversized blocks with 80-char overlap, preserving the source page.
- **Embed & store** — embed all chunk texts, `createMany` the chunks with their JSON `embedding` , set the document READY with page/chunk counts.

Retrieval (`src/lib/retrieve.ts`)

Embed the question, load the user's candidate chunks (status READY, optionally one document), score each by cosine similarity, sort desc, return the top- `k` (default 5).

Generation (`src/app/api/ask/route.ts`)

> **A streamed response with a custom wire format.** The route returns a `ReadableStream` that writes `JSON.stringify({citations})` + a delimiter (`\n<<<ANSWER>>>\n`) + the streamed answer text — so the client can render the **Sources panel immediately**, before the answer finishes. Provider tiering:
>
> 1. **Claude** (`messages.stream` , `claude-sonnet-5`) if a key is set;
> 2. else **Gemini** (SSE `streamGenerateContent`);
> 3. else / on error the **extractive fallback** returns the top-3 passages verbatim, so retrieval is demonstrable even with no API key.
>
> A system prompt instructs the model to answer **only** from the numbered sources, cite inline with `[n]` , and say "I couldn't find that in your documents." if absent. Every Q&A is logged to the `Query` table with its citations and model tag.

3 · Likely interview questions

Q: Why local embeddings instead of an API?
Zero cost, no rate limits, works offline, and no document text leaves the box for the embedding step. all-MiniLM-L6-v2 is small (~25 MB) and fast enough at this scale; it runs in-process via transformers.js.

Q: Why no vector database?
At small scale, storing 384-dim vectors as a JSON column and doing in-memory cosine over a user's chunks is simple and correct. The honest scaling answer is to move to a native `VECTOR` column with an ANN index (e.g. pgvector).

Q: How do citations map to real pages?
PDFs are parsed page-by-page and each chunk records its source page, so a retrieved chunk carries "document + page", which the UI shows as the citation.

Q: Why the custom wire format?
So the Sources panel can render the instant retrieval finishes, while the answer is still streaming — better perceived latency and the user sees the grounding first.

Q: How do you prevent hallucination?
The prompt hard-constrains the model to the numbered sources and to admit when the answer isn't present; and the extractive fallback returns verbatim passages if generation fails.

Honest notes: the README/badges say MySQL 9 + Claude, but the live config is **PostgreSQL + Gemini** (the Anthropic key is empty, a Gemini key is set), and the committed migration is MySQL-dialect. There is also no `.env.example` despite the README's `cp` step.